When Do Language Servers Help Coding Agents?

A Preprint

Ian Barber
Streams Project (affiliation placeholder)

July 2026

Abstract

Three questions organize the experiments: does a language server help a coding agent resolve the correct target, does it make the agent more efficient, and when does checker feedback improve outcomes?

Resolution: little, as long as types are readable in source — which is where the value actually sits. A capable model resolves dispatch-ambiguous targets by reading the receiver's type wherever it appears in visible source, then opens the right file directly; live go-to-definition is token-neutral (ratios 0.94–1.07) across an ablation ladder that moves the type from annotation to construction site to factory indirection. Hide the type-bearing source so the type must be retrieved, and text resolution drops from 15/15 to 12/15 while definition lookup reaches 14/15: retrieval is where the server earns its keep. Sound types also sharpen the resolver itself, distinguishing the correct implementation among 8–15 same-named overrides where erased lookup cannot, though in the agent pilot that precision changed no outcome.

Efficiency: yes, when the span substitutes for reading — and that is the hard part. Compact definitions cut total tokens 3.5–4.7x against whole-file retrieval across three models, and 1.30x at unchanged 11/11 success against a capable grep-plus-ranged-read interface. The gain is conditional on two agent behaviors. The agent must elect the compact operation, which prompting delivers on capable models and training on a 7B. But election is not enough: a pushed span is reread on 35 of 36 instances across Qwen3.6-27B, Sonnet 4.5, and DeepSeek v3.1, an explicit sufficiency instruction removes only 2 of 36, and even a self-elected, provably-usable span is still reread on every instance the local 27B elected and on roughly half the elected instances of two further frontier API models. Training does what neither delivery mode nor instruction can: relabel-tuning the 27B on 39 demonstrations removes the reread on 11 of the 11 held-out instances where it occurred and cuts matched-success tokens 1.59x.

Checking: deliver it late. On twelve identical seeded defects, checker delivery at revision or at submission raises accepted type-clean and held-out-correct outcomes from 1/12 to 10/12 and 11/12, while after-every-edit delivery changes nothing. The submission gate is the one to build: it rejected 10 of 12 bad submissions and every rejection completed the repair, retest, resubmit, accept cycle, at zero false rejections and zero extra cost on twelve matched clean drafts.

The practitioner defaults follow directly: start with text search and ranged reads; add typed resolution where the type must be retrieved rather than read; prompt for span election but train for substitution, and verify the span actually replaces the read; run the checker as a submission gate with a repair loop. Each section below states the finding, the evidence, its strength, and its practical limits. The claim ledger maps every material claim to artifacts and status; the manifest records hashes, configurations, and provenance warnings.

1. Can a language server improve correct resolution?

Finding: for a capable agent, semantic navigation adds almost nothing over text search — because the model reads type information in the visible source and self-localizes. Readable types, not navigation, are the

load-bearing input. Default to grep plus ranged reads. Typed resolution earns its keep in one tested regime: when the type is not readable in source the agent already has and must be retrieved.

A 15-task dispatch suite compares grep/ranged reads with live Pyrefly goto, on tasks where a typed receiver calls one of roughly ten same-named overrides and exactly one is buggy. On the annotated 27B variant, grep, neutral goto, and framed goto solve 15/15, 14/15, and 15/15; matched-success token ratios are 0.972 and 1.041 (C8, C9).

The receiver-type ablation ladder explains why. Moving the type from a call-site annotation (L0) to the test's construction site (L1) to factory indirection (L2) leaves grep-based cost essentially flat — 1,436 → 1,429 → 1,465 mean input tokens at 14–15/15 resolution — and goto stays neutral (0.945–1.065) at every rung, including L2, where only the language server can statically resolve the type. Trajectories show the mechanism directly: the model barely greps, reads the receiver's type where it appears, and opens the one buggy override file first. It is readable type information, not the annotation specifically, that carries resolution: stripping the annotation costs nothing because the type is still readable at the construction site (C9, log).

The typed/erased navigation pilot isolates the resolver mechanism. Typed and erased repositories are byte-identical except one stub: sound per-key `Literal` overloads versus a base return type. Each task has 8–15 same-named overrides and neither prompt nor failure names the target. Mechanical checks confirm typed lookup reaches the gold override, erased lookup returns a non-discriminating base result, widening the key removes the discrimination, and both variants are type-clean (C15). So the precision gain is real at the resolver level. At the agent level it bought nothing in the two-task pilot: all 12 task-condition runs pass (correctness at ceiling), the typed automatic/textual token ratio is 1.037 (task bootstrap 0.988–1.093), every automatic result is followed by a target-file read, and composed lookup adds about six seconds per task (C24).

Hide the type and text resolution starts to fail. Every ladder rung above still showed the type-bearing source in the prompt, testing type location within readable source rather than the cost of retrieving a type. A fourth rung closes that gap: app.py and the test source are hidden, the receiver construction is redacted, and only the use-site line and column remain, so the agent must fetch the type itself. Here the language server improves resolution:

Hidden-type arm	Resolved	Mean input tokens (resolved)	Greps	Definition calls	Whole-file reads
Grep plus ranged reads	12/15	1,390	2.9	0.0	2.1
Definition available	13/15	1,454	1.7	0.4	2.0
Definition framed	14/15	1,349	0.7	1.0	0.9

Text resolution falls from 15/15 in the visible regime to 12/15, and the action mix flips from reading the type once to a genuine hunt: greps rise from 0.3 to 2.9. Two tasks are outright localization rescues — on `record_to_dict` grep thrashes to failure at 2,775 tokens while one definition call resolves it at 1,256, and `resource_cost` fails under grep but resolves under both definition arms. The cost of hiding the type lands on resolution, not tokens: matched-success ratios stay near one (1.009 and 1.051) and cost on tasks solved in both regimes is flat (hidden/visible 1.016). One task (`job_priority`) fails in all three arms from an edit error after correct localization, and the available-definition arm loses one task to thrash.

Strength of evidence. Navigation's near-neutrality holds descriptively across the dispatch suite and three ladder rungs; the resolver-precision mechanism is mechanically verified. The hidden-type advantage rests on one 15-task grid at a single seed with arms separated by one to two tasks, so read it as a direction with two concrete rescues behind it rather than a powered estimate. The automatic-delivery pilot showed no agent-level gain from typed resolution.

Practical limits. Incorrect annotations, `Any`-heavy boundaries, and dynamic dispatch are outside the tested conditions. The hidden regime withholds source a real agent could usually open, so it marks where value appears rather than a typical operating point.

2. Can a language server make an agent more efficient?

Finding: compact definition spans are cheaper than reading — 3.5–4.7x against whole-file retrieval and 1.30x against an efficient grep-plus-ranged-read interface at equal success. The saving exists only when the span substitutes for the read, and substitution is the hard part. Electing the operation is necessary but not sufficient: across a local 27B and two frontier API models, a pushed span is reread almost universally, an explicit "the span is sufficient" instruction barely helps, and even a self-elected, provably-usable span is read past most of the time. The one reliable lever is training — a relabel run removes the reread outright.

Against whole-file retrieval, on the `effic` and `effic_real2` tasks, every attempt in both arms succeeds and compact definitions win by a wide margin (C1):

Model	Pooled input-token ratio	Tasks cheaper with definition	Mean task ratio (95% bootstrap CI)	Success
Qwen3.6-27B	3.50x	11/11	4.02 (2.85–5.30)	44/44 both arms
Claude Sonnet 4.5	3.65x	11/11	3.65 (2.72–5.20)	44/44 both arms
DeepSeek v3.1	4.70x	10/11	6.03 (2.32–10.82)	44/44 both arms

Task-level medians are 3.83, 3.01, and 2.70; DeepSeek's distribution is strongly skewed. Seeds are repeated runs within tasks, and intervals resample task means. The 44 attempts per model span four seeds; the seed-2/3 source shards are not retained, so those cells rebuild from the merged files rather than per-seed originals.

The harder counterfactual keeps grep, ranged reads, and a whole-file fallback available. On the same eleven tasks, pinned Qwen3.5-27B at temperature zero succeeds 11/11 with both interfaces (C27):

Interface	Mean total tokens	Mean retrieval characters	Whole-file reads	Success
Grep plus ranged reads	1,602	1,640	4	11/11
Compact definition plus text fallback	1,235	1,264	0	11/11

The paired ratio is 1.297 text/definition (task-bootstrap 95% CI 1.093–1.527); definitions are cheaper on 10/11 tasks (the exception, `reduceby`, costs 1,670 text versus 1,959 definition — a favorable default, not per-task dominance). A three-task whole-file pilot under the same model produces a 5.67 ratio (2.97–8.96), showing why whole-file-only comparisons overstate the advantage relevant to capable shell agents. A live-first Pyrefly suite reduces mean input from 2,894 to 689 tokens and raises success from 14/24 to 24/24, but the rows do not record which backend answered, so the estimate applies to the composed integration (C4).

The saving is conditional on two agent behaviors:

- Election — the agent must choose the compact operation. The wild 7B elects it near 0%; relabel training raises use to 100% and cost-reward training reaches 86% with 36/36 success, while mean input falls from 3,191 to 684 tokens (C2). Capable models need no training: the 27B elects the definition when offered, and in a real bash agent (mini-swe-agent, Claude Sonnet 4.5) strong system-prompt framing raised codenav use from 0 to 7 and 0 to 5 calls on two of three tasks where mild advertisement did not (one seed, third task noisy) (C3, C6, log).
- Substitution — the agent must not read the file anyway. In one elective retrieval suite the model followed 0 of 11 definitions with a reread, which is where the 1.30x comes from — but that clean substitution is specific to that suite and model, not a general property. Substitution fails across every delivery mode tested. Pushed automatically, a 12-instance replication across a local 27B, Claude Sonnet 4.5, and DeepSeek v3.1 rereads the span almost everywhere, and an explicit "the span is the complete definition, do not open the file" instruction barely helps:

Model	Neutral prompt	Plus explicit "the span is sufficient, do not open the file"
Qwen3.6-27B	11/12 reread	12/12 reread (+294 mean tokens)
Claude Sonnet 4.5	12/12 reread	12/12 reread
DeepSeek v3.1	12/12 reread	10/12 reread

The instruction removed the reread on 0 of 12 tasks for the 27B and Sonnet and only 2 of 12 for DeepSeek. Self-elected — a separate run (Qwen3.6-27B, GLM-4.6, and GPT-5.6 "Luna") where the identical, mechanically-verified usable span is returned only when the model asks for it — is no more reliable: election reduces the reread on the two API models but eliminates it on neither, and the local 27B rereads on every instance it elected (C34).

Model	Pushed: reread	Elected: reread (among elected)
Qwen3.6-27B	11/12	9/9
GLM-4.6	12/12	4/9
GPT-5.6 "Luna"	12/12	7/12

So neither delivery mode nor instruction buys substitution: pushing guarantees a reread, and even a self-elected, provably-usable span is read past most of the time. Training is the lever that works. A DAgger-style relabel [7] run on the 27B, harvesting 39 demonstrations in which the model's own post-span action is redirected from the reread to the edit, then LoRA-tuning on them, removes the behavior on held-out instances drawn from disjoint seeds and disjoint templates (C33):

Qwen3.6-27B, 12 held-out instances	Reread after span	Mean input tokens	Reads per task	Held-out pass
Untrained	11/12	1,157	3.42	11/12
Substitution-trained	0/12	748	0.08	10/12

Every reread that occurred was removed (11 of 11; none induced), and on the nine tasks both arms resolve, tokens fall from 1,493 to 938 — a 1.59x saving, about 555 tokens per task. The contrast with instruction is stark on the same behavior and model: prompting removed 0 of 12 and added 294 tokens per task. During harvest the redirect fired on 46 of 48 rollouts, replicating the reread on fresh templates before any training. Held-out correctness moves 11/12 to 10/12: localization stays intact (zero wrong-file edits, correct first-edit path 12/12), but two `xor` tasks now pass the visible test and fail the held-out oracle, which reads as partial-spec overfit. At one seed that difference is not distinguishable from noise, though it is the direction to watch — and the training set contained no read-required instances, so "elect the span when sufficient, read when it is not" is untested.

In the real bash agent, elicited election likewise did not lower per-call input tokens, partly because a capable shell agent almost never takes the whole-file read the big ratios are measured against — 0–3 whole-file reads in 44–60 actions, with retrieval dominated by grep plus ranged `sed` (C6).

Strength of evidence. The efficiency effect holds in the controlled suite against both the whole-file and the efficient text baseline. Election as a policy lever is supported per-model — training for the 7B, prompting for capable models. Failure to substitute is supported across pushed, instructed, and self-elected delivery on a local model plus two frontier API models at n=12 instances each; the local elective contrast is the clean within-instance one, while the API elect arm delivered an identical usable span but records thinner counters. The training fix is supported on held-out instances disjoint in seed and template from the harvest set, at one model and one seed, with the untrained control reproducing the frozen baseline byte-for-byte.

Practical limits. Tasks are constructed and the headline uses a static AST resolver; real-repository substitution frequency and live-LSP latency are unmeasured. In real shell agents the whole-file counterfactual barely occurs, so the achievable ceiling is the 1.3x regime, not 3–4x — and reaching even that requires the agent to actually substitute, which took a favorable suite or a trained policy.

3. When does checker feedback improve outcomes?

Finding: deliver the checker late. On identical seeded defects, checker delivery at revision or at submission lifts accepted-correct outcomes from 1/12 to 10/12 and 11/12; after-every-edit delivery changes nothing. Prefer the submission gate: it is the only arm that costs nothing on clean work, because it delivers a diagnostic only when a submission is actually defective.

A four-arm grid (C32) runs the same twelve seeded defect/clean pairs — eight defect families, each defect coherent, visible-passing, held-out-failing, with exactly one target-scoped semantic diagnostic — through control (no checker), after-every-edit feedback, one-shot diagnostics at revision, and a v6 submission gate. Control and gate share identical model-generated prefixes through the first completion. Qwen3.5-27B, temperature zero, one rollout per cell:

Defect cohort (n=12)	Control	After every edit	One-shot at revision	Submission gate
Accepted type-clean and held-out-correct	1/12	1/12	10/12	11/12
Bad completion accepted	11/12	11/12	2/12	1/12
Rejected → repaired → retested → resubmitted → accepted	—	—	—	10/10
Mean revision tokens	787	754	1,210	1,380
Mean revision tokens, matched clean cohort	591	651	619	591
False rejections on clean drafts	—	—	—	0/12

Task-bootstrap effects over all 24 workspaces put revision delivery at +0.375 [+0.250, +0.500] and the gate at +0.417 [+0.292, +0.500] against control, with after-every-edit at +0.000 [−0.125, +0.125].

The gradient is a step, not a slope. Both late arms land at 10–11/12 and both early arms at 1/12. The gate's one-task edge over revision delivery comes from refusal, not information: on `auth_pipeline_handler` the model receives the diagnostic and submits anyway with zero edits, and only the gate's rejection produces a repair. Where a rejection occurred, recovery was complete — 10 of 10 rejections ran the full repair, retest, resubmit, accept cycle, and the remaining two defects self-repaired before submission.

The gate is free on clean work; revision delivery is not. Clean drafts cost 591 tokens under both control and gate, since a passing submission draws only a checker call and about 135 ms of latency. One-shot delivery taxes every clean draft (619 tokens, +233 ms) because it hands over a diagnostic unconditionally. That asymmetry, more than the one-task outcome difference, is the argument for submission-time delivery.

Why in-loop delivery does nothing here. The after-every-edit channel barely fired: the checker ran in 1 of 12 defect rows because the model edits in only 1 of 12 before submitting, reading and testing the frozen draft instead (mean edits 0.08 versus 0.17 in control). So this is a null with a largely unexercised mechanism, not a replication of the earlier authoring-suite arm in which the 7B reached 6,367 tokens and its worst held-out score under volunteered feedback (C11).

A type-clean gate is behaviorally blind. The gate's single miss is instructive: on `auth_shapes_protocol` the model self-repairs, the repair clears the seeded type error, the gate sees no diagnostics and accepts — and the held-out test still fails. A checker gate can only be as behaviorally sound as the checker behind it.

Opportunity remains the binding constraint. Capable models rarely leave checker-detectable defects at all: frontier inference and 27B authoring arms sit at ceiling (18/18 and 12/12), 0/3 natural 7B drafts are coherent, and the 2/8 coherent 14B drafts are already type-clean (C10, C16, C22). These defects are seeded precisely because the natural rate is low.

Strength of evidence. The phase gradient is now tested within one design on identical defects at n=12 pairs, one model and one seed, with bootstrap intervals excluding zero for both late arms. Protocol integrity

checks pass: 106 completion events all model-origin, 24/24 identical control/gate first-completion prefixes, zero serialization failures. This supersedes the earlier three-pair pilot (C29) and the two-workspace case series that found no outcome effect from revision delivery (C26) — at n=2 that arm was simply underpowered. The in-loop arm is a null whose channel fired once, so it neither confirms nor refutes the authoring-suite harm.

Practical limits. Defects are constructed and one per workspace, so natural prevalence, population false-rejection rate, and draft-plus-revision cost remain unmeasured. Implement the gate where submission already has a boundary — a completion hook — and measure false rejections, resubmission, accepted-correct yield, and total cost on your own workload.

Execution feedback and external validity

Execution feedback is ceilinged in the committed small-task suite: two frontier models, 14 tasks, three seeds, three delivery modes, 252/252 attempts pass (C12). This does not establish equivalence outside small simulable functions.

The bounded real-repository scan found no fully admissible task. One Django case has a working environment and substantial override ambiguity, but leakage and fix-site resolution were not fully audited; both recorded arms pass and the semantic tool is not elected. Constructed tasks therefore provide the causal apparatus here, and population validity remains open (C17). Audit and rejection reasons: docs/external_validity_recon.md.

Decision checklist

Before adding a language-server integration, check:

1. Is there a concrete bottleneck the server removes? Ambiguous binding, broad retrieval, or repairable static error.
2. Does the integration change what the agent does? Fewer reads, correct target, or repaired defect.
3. Is the outcome better or cheaper? Held-out tests pass, total tokens fall, or wall time drops.
4. Have you measured the full failure path? Gates need rejection, repair, and resubmission events; diagnostics need coherent, repairable drafts.

Practitioner guide

When you see…	Then default to…	Evidence in this report	Verify by measuring…	Do not use when…
The binding is local, visible, and unique	Text search plus ranged reads	Navigation is near token-neutral when readable source exposes the type (C6, C9)	Wrong-file edits, target reads avoided, total tokens	The prompt already names the target file and line
The receiver's type is not readable in available source and must be retrieved	Typed semantic resolution	Text resolution falls to 12/15 while framed definition lookup reaches 14/15, with two outright localization rescues (C30)	Resolution rate first, tokens second — the gain is correctness, not cost	The type is visible somewhere the agent already reads
Overloads, inheritance, factories, or re-exports make the target ambiguous	Typed semantic resolution	Sound types improve resolver precision; agent-level benefit remains open (C15, C24)	Wrong-target work prevented, retrieval reduced, not just tool calls	Text already identifies the exact target
A compact result can replace search and reading	Definition or enclosing-method span	1.30x fewer total tokens vs. grep plus ranged reads at 11/11 success, zero rereads (C27); whole-file contrasts are 3.5–4.7x (C1)	Retrieval-response bytes, total tokens, and post-definition rereads	The agent still performs the same search and reads after receiving the span

When you see…	Then default to…	Evidence in this report	Verify by measuring…	Do not use when…
The agent rereads the file after receiving a span	Train the substitution; election and instruction do not fix it	Rereads occurred on 35/36 pushed spans and instruction removed 2/36; a self-elected usable span was still reread 9/9 (local 27B) and 4/9, 7/12 (API models); relabel training removed 11/11 and cut tokens 1.59x (C31, C33, C34)	Post-span reads of the defining file, and net tokens after the span cost	You cannot train the policy — then expect the span to be additive context, not a saving
A useful service is available but rarely chosen	Improve tool description or policy; train small models	Prompting lifts election on capable models; training lifts a 7B from ~0% to 100% (C2, C3)	Higher election retains correctness and lowers total cost	The service has not first shown value when available
A coherent patch contains checker-detectable errors	Deliver the checker late — at revision, or better at completion with a repair loop	Late delivery lifts accepted-correct from 1/12 to 10–11/12; after-every-edit delivery moves nothing (C32)	Diagnosed-location edits, held-out pass, and diagnostic cost on clean drafts as well as defective ones	The draft is incoherent or the error is not repairable
Bad submission prevention	Staged acceptance gate with explicit repair and resubmit	10/12 bad submissions rejected and 10/10 repaired, resubmitted, accepted; 0/12 false rejections, and no token cost on clean drafts (C32)	False rejections, resubmission, accepted-correct yield, total cost	Abstention is unacceptable or rejection precision is unknown

Related work

Prior work studies missing context, automatic delivery, coherent drafts, and generation constraints — different points on the same resolve-compress-validate chain.

Work	Relevant lesson
Typed Holes [2] and LSPRAG [3]	Push types, definitions, or references when context is missing; automatic delivery removes election failure.
STALL+ [5]	Static analysis effects vary by language and integration phase; retrieval and semantic analysis can complement each other.
CoCoGen [1]	Check a coherent draft, then retrieve context to repair project/API mismatches. This is the opportunity-conditioned regime absent from after-every-edit feedback.
CodeStruct [4]	Structured reads, edits, and syntax validation improve actionability through program structure rather than type information.
CompCoder [8], type-constrained generation [6], and RLCSF v2 [9]	Compiler and server signals can rank, constrain, or train candidates even when repair-time feedback is not useful.

Across this report and prior work, static tooling helps when it supplies missing, correct, compact, actionable information at the phase where the model can use it. The common failure modes are duplicate information, no checker opportunity, ignored or noisy delivery, and cost that exceeds the work saved.

Reproducibility and limits

From a clean clone:

```
python3 -m pip install -e '.[dev,analysis]'
python3 scripts/analysis/reproduce_all.py
```

The reproducer verifies the manifest, reruns retained analyzers, recomputes task-level effects, and executes the navigation manipulation checks without model or API calls. Model runs are separate; see evidence/protocols.md before using the navigation or checker run scripts.

Beyond the per-section limits, two apply throughout: static tooling does not address dynamic behavior, incorrect annotations, `Any`-heavy boundaries, or logic outside the checker; and some retained artifacts contain provenance gaps — unavailable seed shards, incomplete server backend/version records, source-hash mismatches, and invalidated runs — with exclusion reasons and discrepancies reported in the claim ledger and manifest.

Conclusion

For a capable coding agent, the language server's commonly assumed benefits mostly do not survive contact with a text-capable baseline: the model resolves targets by reading types in source, and it retrieves cheaply with grep and ranged reads. What survives is specific, and each piece is conditional. Semantic resolution earns its keep where the type must be retrieved rather than read — hide the type-bearing source and text resolution degrades while typed lookup holds. Compact spans save real tokens when they substitute for reading, and substitution turns out to be a policy you train rather than one you request: models given a pushed span read the file anyway, an explicit instruction does not stop them, even a self-elected span is usually reread, and only relabel training does. Sound types sharpen resolution mechanically, with agent-level value still unproven. And the checker pays off when it arrives late — at revision or, better, as a submission gate with a repair loop, which charges its cost only to defective submissions — rather than as commentary during authoring.

A theme runs through all three: the value is in the types being present and correct in the source, more than in the server that queries them. That is what lets the model self-localize, and keeping it true is the job a checker gate does.

The deployment standard is the same for each service: identify a real opportunity, verify that the semantic signal changes the agent's action, and measure whether that change lowers cost or improves correctness. Availability and invocation are not outcomes. The useful integration is the smallest targeted operation that removes work or prevents an error the agent would otherwise carry forward.

References

[1] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. Iterative refinement of project-level code context for precise code generation with compiler feedback. In Findings of the Association for Computational Linguistics, 2024. CoCoGen; arXiv:2403.16792.

[2] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. Proceedings of the ACM on Programming Languages, 8(OOPSLA2), 2024. arXiv:2409.00921.

[3] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: Lsp-guided rag for language-agnostic real-time unit test generation. arXiv preprint arXiv:2510.22210, 2025.

[4] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. arXiv preprint arXiv:2604.05407, 2026.

[5] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting llm-based repository-level code completion with static analysis. arXiv preprint arXiv:2406.10018, 2024.

[6] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. Type-constrained code generation with language models. In Proceedings of the ACM on Programming Languages (PLDI 2025), volume 9, pages 601–626, 2025. arXiv:2504.09246, doi:10.1145/3729274.

[7] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011. PMLR v15:627–635.

[8] Xin Wang, Yasheng Wang, Yao Wan, Fei Mi, Yitong Li, Pingyi Zhou, Jin Liu, Hao Wu, Xin Jiang, and Qun Liu. Compilable neural code generation with compiler feedback. In Findings of the Association for Computational Linguistics, 2022. CompCoder; arXiv:2203.05132.

[9] Yifan Zhang and Lanser Contributors. Reinforcement learning from compiler and language server feedback. arXiv preprint arXiv:2510.22907, 2025. RLCSF v2 / Lanser-CLI; process reward and replay-bundle methodology.